

```
In [49]: # =====
# Notebook setup
# =====
%load_ext autoreload
%autoreload 2

# Retrain or use existing models
use_cached_weights=True

# Control figure size
figsize=(14, 3.5)
figsize_narrow=(14, 2.5)

import numpy as np
from util import util
import warnings
warnings.filterwarnings("ignore", ".*Consider increasing the value of the `r
warnings.filterwarnings("ignore", ".*GPU available but not used.*")
warnings.filterwarnings("ignore", ".*Previous log files in this directory wi
warnings.filterwarnings("ignore", ".*Checkpoint directory.*exists and is not

# Problem construction
seed, nitems = 42, 20
rel_req = 0.6
rel_buffer_cost = 10
data_tr = util.generate_costs(nsamples=350, nitems=nitems, seed=seed, noise_
prb = util.generate_2s_problem(nitems, requirement=rel_req * data_tr.mean().
data_ts = util.generate_costs(nsamples=150, nitems=nitems, seed=seed, sampli

# Retrieve the previous PFL and SFGE models
pfl = util.TwoPhaseModel(input_size=1, output_size=nitems, hidden_sizes=[],
pfl, history, metadata = util.train_ml_model(pfl, data_tr.index.values.resha

sfge = util.DFLModel(input_size=1, output_size=nitems, hidden_sizes=[], prob
sfge, history, metadata = util.train_ml_model(sfge, data_tr.index.values.res

pfl_nl = util.TwoPhaseModel(input_size=1, output_size=nitems, hidden_sizes=|
pfl_nl, history, metadata = util.train_ml_model(pfl_nl, data_tr.index.values

sfge_nl = util.DFLModel(input_size=1, output_size=nitems, hidden_sizes=[16],
sfge_nl, history, metadata = util.train_ml_model(sfge_nl, data_tr.index.valu
```

The autoreload extension is already loaded. To reload it, use:  
 %reload\_ext autoreload

## BB Differentiation via Importance Sampling

### Dusting Off Some Old Tools

SFGE is not the only technique to take advantage of stochastic smoothing

Let's start again from the smoothed regret definition:

$$\tilde{L}_{DFL}(\theta) = \mathbb{E}_{(x,y) \sim P(X,Y), \tilde{y} \sim P(\tilde{Y};\theta)} [\text{regret}(y, \tilde{y})]$$

where  $P_{\tilde{y}}(\tilde{Y}; \theta) = \mathcal{N}(h(x; \theta))$ , for sake of brevity

**Let's assume we got our samples from a *different distribution* instead**

In other words, we have that  $\tilde{y} \sim Q(\tilde{Y})$

- We will consider how we can estimate the above expectation
- ...Based on the "wrong" samples we have

**This is actually a problem we have already encountered!**

## Bridging Distributions via Importance Sampling

**With our samples we can directly estimate:**

$$\mathbb{E}_{(x,y) \sim P(X,Y), \tilde{y} \sim Q(\tilde{Y})} [g(\tilde{y}; \theta)]$$

with:

$$g(\tilde{y}; \theta) = \frac{p_P(\tilde{y}; \theta)}{p_Q(\tilde{y})} \text{regret}(y, \tilde{y})$$

By combining the two together we get:

$$\mathbb{E}_{(x,y) \sim P(X,Y), \tilde{y} \sim Q(\tilde{Y})} \left[ \frac{p_P(\tilde{y}; \theta)}{p_Q(\tilde{y})} \text{regret}(y, \tilde{y}) \right]$$

## Bridging Distributions via Importance Sampling

**By expanding the expectation we get:**

$$\mathbb{E}_{(x,y) \sim P(X,Y), \tilde{y} \sim Q(\tilde{Y})} \left[ \frac{p_P(\tilde{y}; \theta)}{p_Q(\tilde{y})} \text{regret}(y, \tilde{y}) \right] = \quad (1)$$

$$\mathbb{E}_{(x,y) \sim P(X,Y)} \left[ \int_{\tilde{y}} p_Q(\tilde{y}) \frac{p_P(\tilde{y}; \theta)}{p_Q(\tilde{y})} \text{regret}(y, \tilde{y}) d\tilde{y} \right] = \quad (2)$$

$$\mathbb{E}_{(x,y) \sim P(X,Y), \tilde{y} \sim P(\tilde{Y};\theta)} [\text{regret}(y, \tilde{y})] \quad (3)$$

Which means that:

- The first expectation, with the *importance weights*  $p_P/p_Q$

- ...Is an estimator for the quantity we are interested in

This is a classical application of *importance sampling*

## Differentiation via Importance Sampling

**This is interesting since we can use it to *obtain a gradient***

- Given a training example  $(x, y)$
- Let's assume we get access to  $N$  samples  $\tilde{y}_k$  from  $Q(\tilde{Y})$

Then we have that:

$$\mathbb{E}_{\tilde{y} \sim Q(\tilde{Y})} \left[ \frac{p_P(\tilde{y}; \theta)}{p_Q(\tilde{y})} \text{regret}(y, \tilde{y}) \right] \simeq \frac{1}{N} \sum_{k=1}^N \frac{p_P(\tilde{y}_k; \theta)}{p_Q(\tilde{y}_k)} \text{regret}(y, \tilde{y}_k)$$

where:

- The probability  $p_Q(\tilde{y})$  is *fixed* and known (we can choose it!)
- The probability  $p_P(\tilde{y}; \theta)$  depends on  $\theta$  and *can be differentiated*

## Differentiation via Importance Sampling

**Overall, at every gradient step:**

1) We draw samples according to sampling distribution  $Q(\tilde{Y})$ :

- For example, we can use  $\mathcal{N}(\hat{y}, \sigma')$
- In our implementation, we will have  $\sigma' = \sigma\rho$ , with  $\rho > 1$

2) We compute the loss function:

$$\frac{1}{N} \sum_{k=1}^N \frac{p_P(\tilde{y}_k; \theta)}{p_Q(\tilde{y}_k)} \text{regret}(y, \tilde{y}_k)$$

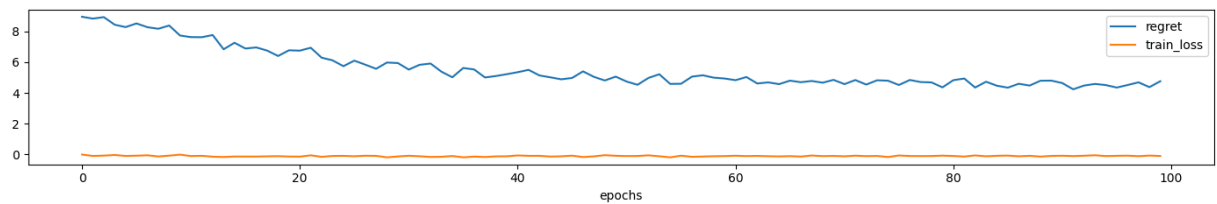
- As in the SFGE case, it makes sense to use a baseline and compute *advantage*
- In our implementation, we will do it by standardizing the regret over the mini-batches

## Basic IS Policy Gradient

**The result is a basic black-box DFL method based on IS**

...Which relies on ideas from RL and multi-armed bandits problems

```
In [50]: bonis_nl = util.DFLModel(input_size=1, output_size=nitems, hidden_sizes=[16])
         bonis_nl, history, metadata_bonis = util.train_ml_model(bonis_nl, data_tr, ir=0.1)
         util.plot_training_history(history, metadata=metadata_bonis, figsize=figsize)
         util.print_ml_metrics(bonis_nl, data_tr.index.values.reshape(-1, 1), data_tr.index.values.reshape(-1, 1))
         util.print_ml_metrics(bonis_nl, data_ts.index.values.reshape(-1, 1), data_ts.index.values.reshape(-1, 1))
```



training time: 34.8687

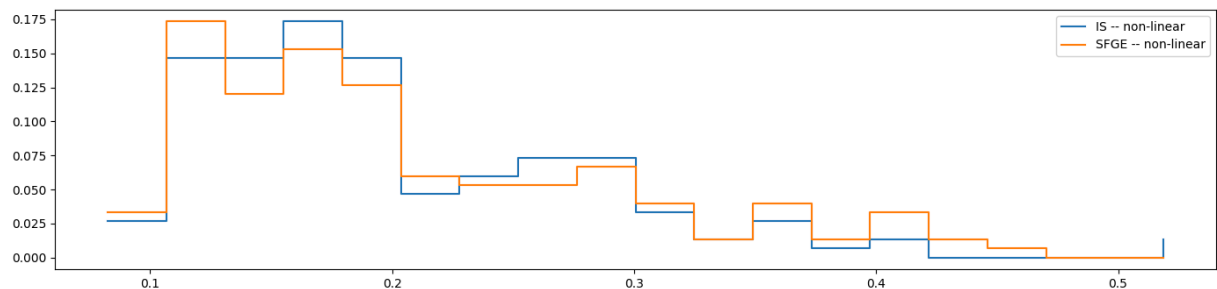
R2: 0.711, MAE: 0.085, RMSE: 0.103 (training)

R2: 0.804, MAE: 0.063, RMSE: 0.078 (test)

## Basic IS Policy Gradient

In terms of performance, it is comparable with the SFGE approach

```
In [51]: bonis_nl_evpf = util.compute_evpf_2s(prb, bonis_nl, data_ts, tlim=10)
         sfge_nl_evpf = util.compute_evpf_2s(prb, sfge_nl, data_ts, tlim=10)
         util.plot_histogram(bonis_nl_evpf, figsize=figsize, label='IS -- non-linear')
```



Mean: 0.203 (IS -- non-linear), 0.208 (SFGE -- non-linear)

The two approaches are actually combined in some algorithms

## Recycling Samples

We can use IS to *reduce our reliance on sampling*

For the *smoothing distribution*, the can make usual choice:

$$P(\tilde{Y}; \theta) = \mathcal{N}(h(x; \theta), \sigma)$$

For the *sample and its distribution*, we use:

$$\tilde{y} \sim Q(\tilde{Y}) = \mathcal{N}(\hat{y}, \sigma)$$

- Where  $\tilde{y}$  is the last evaluated sample
- ...And  $\hat{y}$  is the prediction for its smoothing distribution

**In practice, we can *reuse past samples***

Which is a big deal, since sampling requires solving optimization problems

## ...Until they are Stale

**How much can we push this on?**

Let's consider the variance of our IS-based estimator:

$$\text{Var}_{\tilde{y} \sim Q(\tilde{Y})} \left( \frac{p_P(\tilde{y}; \theta)}{p_Q(\tilde{y})} \text{regret}(y, \tilde{y}) \right)$$

By basic properties of variance, we can expand it as:

$$\mathbb{E}_{\tilde{y} \sim Q(\tilde{Y})} \left[ \frac{p_P(\tilde{y}; \theta)^2}{p_Q(\tilde{y})^2} \text{regret}(y, \tilde{y})^2 \right] - \mathbb{E}_{\tilde{y} \sim Q(\tilde{Y})} \left[ \frac{p_P(\tilde{y}; \theta)}{p_Q(\tilde{y})} \text{regret}(y, \tilde{y}) \right]^2$$

Then, by relying on our previous results we get:

$$\mathbb{E}_{\tilde{y} \sim Q(\tilde{Y})} \left[ \frac{p_P(\tilde{y}; \theta)^2}{p_Q(\tilde{y})^2} \text{regret}(y, \tilde{y})^2 \right] - \mathbb{E}_{\tilde{y} \sim P(\tilde{Y}; \theta)} [\text{regret}(y, \tilde{y})]^2$$

## ...Until they are Stale

**Let's look again at the last expression:**

$$\mathbb{E}_{\tilde{y} \sim Q(\tilde{Y})} \left[ \frac{p_P(\tilde{y}; \theta)^2}{p_Q(\tilde{y})^2} \text{regret}(y, \tilde{y})^2 \right] - \mathbb{E}_{\tilde{y} \sim P(\tilde{Y}; \theta)} [\text{regret}(y, \tilde{y})]^2$$

- We cannot really control the second terms
- We do not know the regret landscape
- ...But we can definitely evaluate the value of  $p_P(\tilde{y}; \theta)^2 / p_Q(\tilde{y})^2$

**We can therefore define a sampling criterion**

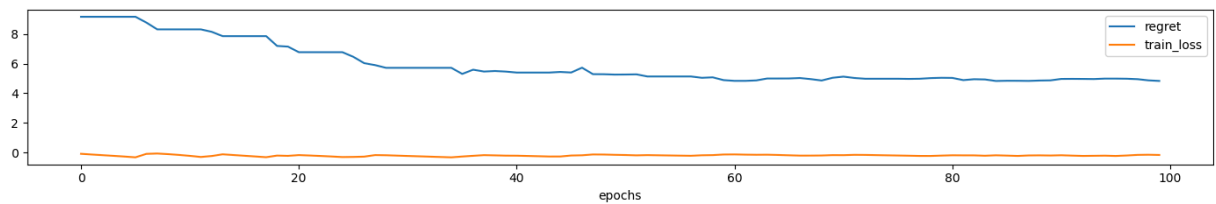
- If the ratio  $p_P(\tilde{y}; \theta) / p_Q(\tilde{y})$  is above a threshold, we sample
- If it is below, we reuse the last available sample

## Basic IS Off-Policy Gradient

**In RL, methods that reuse samples are called "off-policy"**

Our implementation is a bit slower, since parallelization is trickier in this case

```
In [52]: bofis_nl = util.DFLModel(input_size=1, output_size=nitems, hidden_sizes=[16])
bofis_nl, history, metadata_bofis = util.train_ml_model(bofis_nl, data_tr, ir)
util.plot_training_history(history, metadata=metadata_bofis, figsize=figsize)
util.print_ml_metrics(bofis_nl, data_tr.index.values.reshape(-1, 1), data_tr)
util.print_ml_metrics(bofis_nl, data_ts.index.values.reshape(-1, 1), data_ts)
```

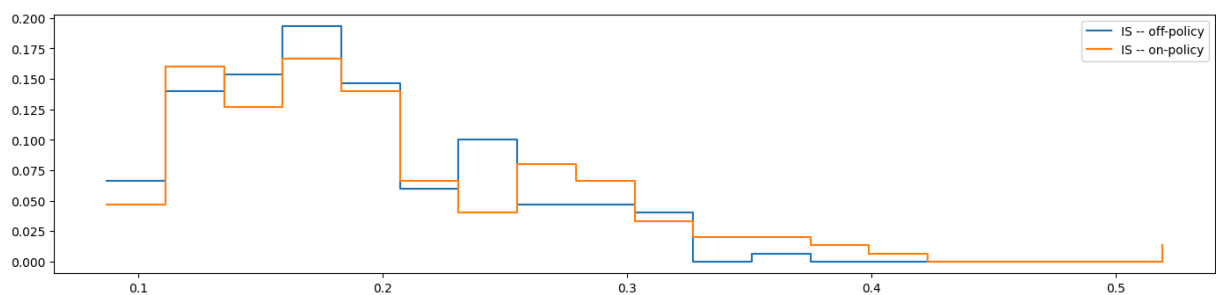


training time: 108.3270  
R2: 0.710, MAE: 0.084, RMSE: 0.104 (training)  
R2: 0.802, MAE: 0.063, RMSE: 0.079 (test)

## Basic IS Off-Policy Gradient

In terms of EVPF, the results are comparable to the previous ones

```
In [53]: bofis_nl_evpf = util.compute_evpf_2s(prb, bofis_nl, data_ts, tlim=10)
util.plot_histogram(bofis_nl_evpf, figsize=figsize, label='IS -- off-policy')
```

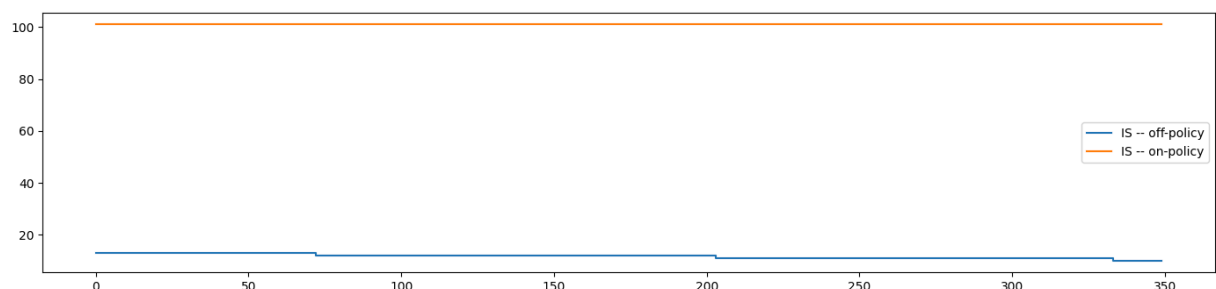


Mean: 0.186 (IS -- off-policy), 0.203 (IS -- on-policy)

## Basic IS Off-Policy Gradient

But the number of computed solution is vastly different

```
In [54]: bofis_counts = list(metadata_bofis['solution count'].values())
bonis_counts = list(metadata_bonis['solution count'].values())
util.plot_solcount(bofis_counts, figsize=figsize, label='IS -- off-policy',
```



Total: 4108 (IS -- off-policy), 35350 (IS -- on-policy)

- The plot shows the number of solved problems, by example, in decreasing order
- In a non-trivial problem, this would dominate the computational cost

## Some Final Remarks

### **DFL is an example of how learning and optimization can be combined**

The two approaches have complementary strengths

- ML methods excel at dealing with uncertainty and non-linearities
- Optimization methods can deal with constraints and search

With a combination, each approach can focus on what it does best

### **For the best results, we need to be clever**

- The trick is thinking about the problem as a whole
- ...Then choosing how to split responsibilities between ML and optimization
- The goal is have two simple, integrated problems, instead of single, complex, one

### **If you are curious, it's worth checking:**

- About these methods, the classical [RL book by Sutton & Barto](#) (especially ch. 5)
- About integrating RL and optimization, [this paper of ours can be a good start](#)